09 · Visits and schedule

Booking a home visit, recording its outcome, and keeping both sides informed.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

A worker books a visit (often straight from a flag). Both the citizen and the worker get an alert. The worker's **Schedule** page groups visits into needs outcome, happening now, today, upcoming and recent outcomes, with buttons for **Mark done**, **Nobody there**, **Move** and **Cancel**. Citizens see their visits on **Alerts & visits**.

A visit's stored **status** is only what someone decided (scheduled, completed, cancelled, missed). Its **state** (upcoming, today, due now, overdue) is worked out from the status and the clock every time, so it is never out of date.

Files involved

Only the files this feature uses.

```
backend/app/
├── services/visit_state.py    visit_state(): status + clock -> state; India time; 4-hour grace
├── services/visits.py         the alert messages: booked, moved, cancelled, missed
├── api/routes/phc.py          POST /phc/visits (book), GET /phc/visits
├── api/routes/schedule.py     GET /schedule (worker), GET /schedule/mine (citizen), PATCH
│                              /schedule/{id}
└── models/scheduled_visit.py  table scheduled_visits
frontend/src/
├── pages/phc/Schedule.tsx     the worker's schedule and VisitRow
├── components/app/phc.tsx     BookVisitForm, useDefaultSlot()
└── services/phc.ts            bookVisit(), getSchedule(), updateVisit()
```

How the pieces connect

1. `BookVisitForm` sends `POST /phc/visits`; the backend checks the catchment, the time and that no other visit is open, then saves it and notifies both sides.
2. If the booking came from a flag, that flag is marked reviewed ("Follow-up visit booked").
3. Loading a schedule first runs `notify_missed`, which sends the one-time "needs an outcome" alerts for overdue visits.
4. Buttons send `PATCH /schedule/{id}` with an action; the backend checks the action makes sense for the visit's state.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/services/visit_state.py` · visit_state

```
43  def visit_state(status: VisitStatus, scheduled_for: datetime, now: datetime | None = None) -> VisitState:
44      now = now or now_utc()
45      if status != VisitStatus.scheduled:
46          return status.value  # type: ignore[return-value]
47      if scheduled_for > now:
48          return "today" if local_date(scheduled_for) == local_date(now) else "upcoming"
49      if now - scheduled_for <= GRACE:
50          return "due_now"
51      return "overdue"
```

What the code means

- If someone already decided (completed, cancelled, missed), that is the state.
- A future visit is `today` or `upcoming`, compared by **local** date (Asia/Kolkata), not UTC.

- A past visit is `due_now` for the first 4 hours (`GRACE`), then `overdue`. Each visit is in exactly one state, so counts never overlap.

2. `backend/app/api/routes/schedule.py` · check_booking_time

```
133 def check_booking_time(moment: datetime) -> None:
134     now = now_utc()
135     if moment <= now:
136         raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, "Choose a time in the future.")
137     if moment > now + BOOKING_HORIZON:
138         raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, "Visits can be booked up to 90 days ahead.
            ")
```

What the code means

- Bookings must be in the future and at most 90 days ahead; otherwise a 422 error with a clear message.

3. `backend/app/api/routes/phc.py` · book_visit

```
845 @router.post("/visits", response_model=VisitOut, status_code=status.HTTP_201_CREATED)
846 async def book_visit(
847     data: VisitCreate, worker: User = Depends(require_catchment_worker), db: AsyncSession = Depends(get_sess
        ion)
848 ) -> VisitOut:
849     citizen = await get_catchment_citizen(db, worker, data.citizen_id)
850     check_booking_time(data.scheduled_for)
851 
852     case_row = None
853     if data.case_id is not None:
854         case_row = await db.scalar(
855             select(ProcessedData)
856             .where(ProcessedData.id == data.case_id, ProcessedData.user_id == citizen.id)
857             .with_for_update()
858         )
859         if case_row is None:
860             raise HTTPException(status.HTTP_404_NOT_FOUND, CASE_NOT_FOUND)
861 
862     existing = await db.scalar(
863         select(ScheduledVisit).where(
864             ScheduledVisit.citizen_id == citizen.id,
865             ScheduledVisit.status == VisitStatus.scheduled,
866             ScheduledVisit.scheduled_for > now_utc() - GRACE,
867         )
868     )
869     if existing is not None:
870         raise HTTPException(
871             status.HTTP_409_CONFLICT,
872             "This citizen already has a visit booked. Move that visit instead of booking a second one.",
873         )
874 
875     visit = ScheduledVisit(
876         worker_id=worker.id,
877         citizen_id=citizen.id,
878         case_id=case_row.id if case_row else None,
879         scheduled_for=data.scheduled_for,
880         reason=data.reason or None,
881         location=data.location or citizen.area,
882     )
883     db.add(visit)
884     # Booking a follow-up is a review: the case leaves the queue.
    ... (9 more lines in the file: saving and notifying follow)
```

What the code means

- `data: VisitCreate` is the validated body (citizen, optional flag id, time, reason, location).
- If a `case_id` (flag) is given it must belong to this citizen; `with_for_update()` locks it while the visit is created.
- Only one open visit per citizen: a second booking gives 409 and asks to move the existing one.
- `location=data.location or citizen.area` defaults the place to the citizen's area.

4. `backend/app/api/routes/schedule.py` · update_visit (outcomes)

```
142 async def update_visit(
143     visit_id: uuid.UUID,
144     data: VisitUpdate,
145     worker: User = Depends(require_catchment_worker),
146     db: AsyncSession = Depends(get_session),
147 ) -> VisitOut:
148     row = (await db.execute(_mine(worker).where(ScheduledVisit.id == visit_id).with_for_update(of=ScheduledV
        isit))).first()
149     if row is None:
150         raise HTTPException(status.HTTP_404_NOT_FOUND, VISIT_NOT_FOUND)
151     visit, citizen = row
152     if visit.status != VisitStatus.scheduled:
153         raise HTTPException(status.HTTP_409_CONFLICT, f"This visit already has an outcome: {visit.status.val
            ue}.")
154
155     now = now_utc()
156     state = visit_state(visit.status, visit.scheduled_for, now)
157     if data.action == "complete":
158         if state == "upcoming":
159             raise HTTPException(status.HTTP_409_CONFLICT, "This visit is on a later day. Move it if it happe
                ned early.")
160         visit.status = VisitStatus.completed
161     elif data.action == "miss":
162         if state not in PASSED:
163             raise HTTPException(status.HTTP_409_CONFLICT, "A visit can only be marked 'nobody there' once it
                s time has passed.")
164         visit.status = VisitStatus.missed
165     elif data.action == "cancel":
166         visit.status = VisitStatus.cancelled
167         notify_changed(db, visit, citizen, worker, "cancelled")
168     else:
169         assert data.scheduled_for is not None
170         check_booking_time(data.scheduled_for)
171         visit.scheduled_for = data.scheduled_for
172         visit.missed_notified_at = None  # a new time earns a new one-time reminder
173         notify_changed(db, visit, citizen, worker, "moved")
174     await db.commit()
175     return visit_for_worker(visit, citizen, now_utc())
```

What the code means

- `_mine(worker)` limits it to the worker's own visits with citizens still in the catchment; `with_for_update(of=ScheduledVisit)` locks the visit row.
- Only a scheduled visit can change; otherwise 409 with its current outcome.
- `complete` is refused for a later day; `miss` ("nobody there") only once the time has passed (`PASSED`).
- `move` checks the new time and clears `missed_notified_at`, so the new time gets its own reminder. Cancel and move both notify the citizen.

5. `backend/app/services/visits.py` · notify_missed

```
49 async def notify_missed(session: AsyncSession, *, worker_id: uuid.UUID | None = None, citizen_id: uuid.UUID
       | None = None) -> int:
50     """Write the one-time 'missed' alerts for overdue visits of one worker or one citizen.
51
52     The UPDATE ... WHERE missed_notified_at IS NULL claims each visit atomically,
53     so two schedule loads racing each other cannot both send it. Commits.
54     """
55     now = now_utc()
56     claim = (
57         update(ScheduledVisit)
58         .where(
59             ScheduledVisit.status == VisitStatus.scheduled,
60             ScheduledVisit.scheduled_for < overdue_before(now),
61             ScheduledVisit.missed_notified_at.is_(None),
62         )
63         .values(missed_notified_at=now)
64         .returning(ScheduledVisit.id, ScheduledVisit.worker_id, ScheduledVisit.citizen_id, ScheduledVisit.sc
            heduled_for)
65     )
66     if worker_id is not None:
67         claim = claim.where(ScheduledVisit.worker_id == worker_id)
```

```
    if citizen_id is not None:
        claim = claim.where(ScheduledVisit.citizen_id == citizen_id)
    claimed = (await session.execute(claim)).all()
    if not claimed:
        return 0
... (24 more lines in the file: the two alert messages follow)
```

What the code means

- One UPDATE both finds overdue visits and marks them as notified (`missed_notified_at = now`), returning the rows it changed (`.returning(...)`).
- Because the claim and the check happen in one statement, two page loads at the same moment cannot both send the reminder.

6. `frontend/src/components/app/phc.tsx` · BookVisitForm (submit)

```
export function BookVisitForm({
  citizen,
  caseId,
  onBooked,
  onCancel,
}: {
  citizen: VisitPerson;
  caseId?: string | null;
  onBooked: () => void;
  onCancel: () => void;
}) {
  const [when, setWhen] = useDefaultSlot();
  const [reason, setReason] = useState("");
  const [location, setLocation] = useState(citizen.area ?? "");
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const submit = async (e: FormEvent) => {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      await bookVisit({
        citizen_id: citizen.id,
        case_id: caseId ?? undefined,
        scheduled_for: new Date(fromLocalInput(when)).toISOString(),
        reason: reason || undefined,
        location: location || undefined,
      });
      onBooked();
    } catch (err) {
      setError(errorMessage(err));
      setBusy(false);
    }
  };

```

What the code means

- `useDefaultSlot()` suggests tomorrow at the start of the worker's working hours (from Settings).
- `e.preventDefault()` stops the browser's normal form submit so the code can send the request itself.
- `new Date(fromLocalInput(when)).toISOString()` turns the local date-time input (India time) into a UTC timestamp for the API.
- `reason || undefined` leaves out empty fields.

7. `frontend/src/pages/phc/Schedule.tsx` · VisitRow (actions)

```
function VisitRow({ visit, onChanged }: { visit: Visit; onChanged: () => void }) {
  const [busy, setBusy] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [moving, setMoving] = useState(false);
  const [confirmCancel, setConfirmCancel] = useState(false);
  const [when, setWhen] = useState(() => toLocalInput(new Date(Math.max(Date.now() + 3600_000, new Date(visi
      t.scheduled_for).getTime()))));

  const act = async (action: "complete" | "cancel" | "miss" | "move") => {
    setBusy(action);
    setError(null);
    try {
      await updateVisit(visit.id, action === "move" ? { action, scheduled_for: new Date(fromLocalInput(when)
          ).toISOString() } : { action });
      onChanged();
    } catch (e) {
      setError(errorMessage(e));
      setBusy(null);
    }
  };

  const passed = visit.state === "due_now" || visit.state === "overdue";
```

What the code means

- `act` sends the chosen action; `move` also sends the new time. `busy` holds which button is working so only that one shows a spinner.
- `passed` controls whether "Nobody there" is offered, matching the backend rule.

Syntax used in this feature

Code	What it means
`update(...).returning(cols)`	UPDATE that also returns the changed rows.
`with_for_update(of=Table)`	Lock only that table's rows.
`e.preventDefault()`	Stop the browser's default action (here, submitting the form).
`new Date(x).toISOString()`	A timestamp as text in UTC.